

①

Basic algorithm Design techniques. we will start with fairly simple problems that many of which you may have already seen but soon will hopefully see solutions to nontrivial problem which you may not have seen.

The first example that we would like to start with is finding the minimum element in the array.

Findmin

Input: An array A

out : minimum element in A

This is the problem which many of you have seen earlier.

The standard solution is you take a temporary variable "temp".

temp: contains the current minimum and is updated in each step.

we start with the first element in the array scan the array element by element. 1st element, 2nd element, 3rd element upto the last element in the array

(2)

each time you compare the element with temp and update temp if necessary. This simple technique is very powerful and used often. and gives rise to a problem solving strategy which is as follows

If you are given the input as an array or list then if you can solve the problem for the 1st $n-1$ elements of the array, can you solve it for the n elements.

This is the step put in a loop. The important step of the solution is this

$$\left[\begin{array}{l} \text{if } A[i] < \text{temp} \\ \text{then temp} \leftarrow A[i] \end{array} \right.$$

This is the ~~crucial~~ ^{crucial} step and usually put in a loop. i varies from 1 through n where n is the no of elements in the array. and at the end of this, temp will have the minimum element in the array.

This design technique is often called
induction (Recursion/Iteration)
go with this

(3)

* The second thing to keep in mind is the ordering

* Store ^{necessary} values already computed

Let us explain each of this

Induction: what we mean by this is

① you want to solve a problem, and the problem has some size say n

② induction says supposing you can solve the problem for smaller values of input, can I extend these solution for ~~for~~ bigger input.

For example suppose if I can solve a problem for all arrays of size $n-1$, can I solve this problem for an array of size n .

Now the step which takes me from $n-1$ to n is the crucial steps and once you come of with this step, we just put it in a loop. or you use recursion. You first recurse on an array of size $n-1$ and then extend it to an array of size n . This is what is known as induction and the base of every algorithm design technique.

Now the second point is the ordering of the input

(4)

Looking the input in the right order often helps. In this case it is simple. you look at the array elements in the increasing order of their index. In fact you could look at the array in any order. It does not matter as long as you look at all the elements. But there are cases where ordering places a very crucial role in solving the problem.

The third thing is to store some of the values that are already computed. In this case it is again simple, you just store the previous minimum, the temporary variable. These are the values that you use in future.

Almost every algorithm design technique that we study here are combinations of these three. Some of them will ^{use} just induction and storing values, some of them will use all three. Some of them will just order the input and use induction. So these are the three things that you must keep back of your mind, when you design algorithm.

Analysis: Every algorithm that we design we analyze it. Analysis is as important a part of this subject as design is. and we would like algorithms to be as fast as possible in the worst case. So all our analysis pertain to worst case.

In the algorithm Findmin that we are looking at. We look at every array element once and make one comparison. This leads us to make $n-1$ comparisons. We do not really compare anything with the 1st element. but with each of the subsequent element we make one comparison. n is the size of the array. So we make $n-1$ comparisons. Along with comparison we also need to store the new value in the 'temp'. But no of time we do this is one more than the no of comparison we make in the worst case. So if we are really not worried about the constant, then we can just focus on the number of comparisons. As long as I bound the no of comparison, the other small operations that I do, like incrementing the index variable of the array or storing the value in temp, they all are in the same order as

(6)

number of comparisons. So the time taken here we would say is $O(n)$. But ~~with~~ focus is on the number of comparisons only.

Now the question that one can ask and one should ask is this the best?

Can I find the minimum element in an array ~~if~~ using less than $n-1$ comparisons?

I guess ~~least~~ most of you will say of course you need $n-1$ comparisons. Without $n-1$ comparisons how can you find the minimum element?

Why is this so? Can you logically argue that you actually need $n-1$. The argument is easy but not absolutely trivial.

Now let us see why you need $n-1$ comparisons: Well every element ~~needs to~~ ^{must} be compared to something or the other; if it is not compared then you may make it minimum. So if you do not compare an element then you can not tell the minimum element in the array.

Now this gives you $n/2$ comparisons assuming n is even, so at least $n/2$ comparisons are needed.

(7)

The reason is that every element must be in some comparison. This can be done in $n/2$ comparsion

1st element is compared with the 2nd element, The third element is compared with the 4th element and so on. So with $n/2$ comparsion if n is even or $n/2+1$ if n is odd, (well $\lceil n/2 \rceil$) comparsion are all necessary to satisfy the condition that every element must be in some comparison.

Now clearly $n/2$ comparsion is not the right answer, $n-1$ is the right answer and why is that? here is an argument.

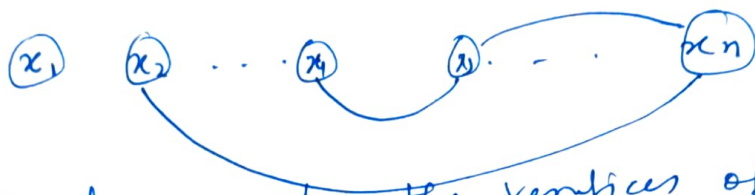
Initially when you have not made any comparison there are n candidates for the minimum. each element of the array can be actually be minimum. Now when you make a comparison you can get rid of only one element of these candidates. whichever is smaller that still remains a candidate. The larger one that ~~can not be~~ no longer ~~be~~ remains the candidate.

we have n elements to start with, I need $n-1$ comparison to ~~find~~ get rid of $n-1$ elment,

(8)

This is ~~the~~ actually the complete proof, though a bit hand wavy. We can make it more rigorous.

Here is one more way of looking at it supposing you draw the following graph. Initially we have all these elements in the array



The elements are also the vertices of the graph. Now when x_i and x_j are compared I draw an edge between them. When I compare say x_2 and x_n I draw another edge as shown. So as I make comparisons I keep drawing these edges.

So once our program ends ^{we} you have done all the comparisons. Now I look at the graph and look at each connected component in this graph. Now if there are more than one ^{connected} component in this graph then I will not be able to tell the minimum.

Each connected component has one minimum but ~~the~~ out of them which one is the global minimum that I can not say.

⑨

So I better have one connected component. This means from the discrete structure class, you need $(n-1)$ edges. So this is the argument that tells us that we need $(n-1)$ comparisons to find the minimum from an array of n elements.

Both the approaches have the same idea behind. In this case you initially start with n connected components and each time you connect an edge you decrease the no. of components by one ~~about~~ 1. So you need $n-1$ comparisons to get a single connected component.

Let us ^{look at} a slight variation of this problem. Now I want to find not just the minimum element in the array but also the Max element. This problem is called MaxMin.

Problem MaxMin

Input: Array A

Output: Max and Min elements in A

Now if I want to find only the Maximum the procedure is clearly the same as finding the min.

But now I want to find both the maximum and the minimum element. well I could first find the minimum and then I could find the maximum

How many comparison does this take?
well $n-1$ for the maximum, $n-1$ for the minimum and this makes $2n-2$.

So the no of comparisons that the naive algorithm takes is $2n-2$. Now we ask the same question: can this be the best?

You can try the previous argument that we had of connected component and you can show that it takes $n-1$ for each of Min and Max. That is fine but anything more than that is very difficult to prove

well it is not absolutely impossible but it is difficult.

Now look at some small values.

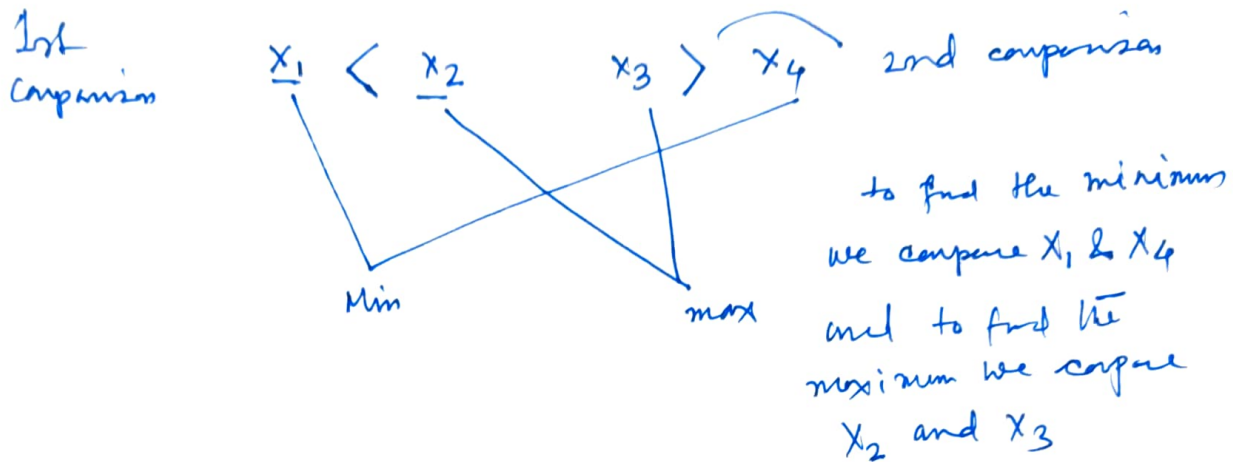
Suppose we have say 4 elements

$x_1 \quad x_2 \quad x_3 \quad x_4$

The naive algorithm would take x_1 compared all of them find the minimum. then we take again x_2 compared all others and maintaining the temporary maximum. and take $2n-2$ comparison.

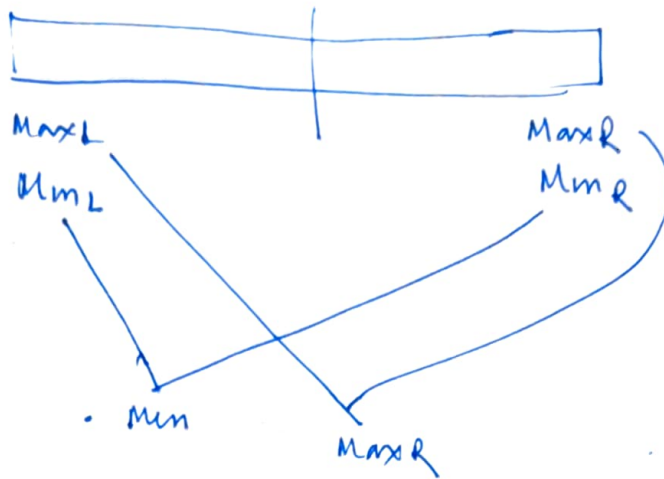
Now many of this comparisons are repeated. Our goal is to get rid of these unnecessary comparisons.

Now in our 4 elements examples, here is what we can do.



Now how many comparisons we have made? we have made 4 comparisons. What does our naive algorithm say? It says $2n-2 = 2 \cdot 4 - 2 = 6$ comparisons we seem to have done certainly better $2n-2$

So this trick can be applied recursively. So when we have larger no of element. We divide the elements in two equal halves, find the min & max of left half and right half. Compare the mins of left half and right to get the actual minimum and similarly the max of both the half are compared to get the actual maximum.



Now how many comparisons does this algorithm take

- Divide int halves (left & right)
- Recurse on both parts
- then output final solution from the solutions of the two part

So I have written down the essence of the algorithm without actually writing the details

$T(n)$: time taken by max min on arrays of size n

$$T(n) = 2T(n/2) + 2 \rightarrow \text{between two mins and two max}$$

$T(2) = 1$ for two elements we can find min and max in 1 comparison
when n is even

so what is the solution to this recurrence? ⁽¹³⁾

$$T(n) = 2 \{ 2T(n/4) + 2 \} + 2$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2^2 + 2$$

...

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + 2^i + 2^{i-1} + \dots + 2$$

$$\text{set } \frac{n}{2^i} = 2$$

because we know
 $T(2) = 1$

$$T(n) = 2^i + 2^i + 2^{i-1} + \dots + 2$$

$$= \underbrace{2^{i+1}} + \underbrace{2^{i-1} + \dots + 2}$$

$$= n + \frac{n}{2} - 2$$

we also know that
 $2^{i+1} = n$

$$= \frac{3n}{2} - 2$$

$$n = 2$$

we get

$$\frac{6}{2} - 2$$

$$= 3 - 2 = 1$$

which is
true

This technique is called divide and conquer